

Agrupamento, Combinação de Consultas, Consultas Aninhadas e Função Rank()

Banco de Dados II

Prof. Me. Jefferson Passerini



Agrupamento



Agrupamento

Definição

- Agrupamento de dados é o processo de combinação de colunas com valores repetidos em uma ordem lógica;
- O agrupamento é feito pela coluna (campo) que será usada como pivô para obter informações da tabela;
- A partir do qual pode-se gerar as estatísticas por meio das funções de agregação.
- Por exemplo:
- *numa tabela escolar saber rapidamente quantos alunos são do sexo masculino e do sexo feminino.*

Sintaxe

- O agrupamento de dados na linguagem de consulta SQL é obtido com a cláusula GROUP BY combinada com o comando SELECT após o uso da cláusula WHERE e antes da cláusula ORDER BY.
- A estrutura sintática da instrução completa passa a ser:
- SELECT [tipo] <campos> FROM <tabela> WHERE [condição]
- GROUP BY <campos>
- ORDER BY <campos>

Agrupamento

Insira novos itens vendidos

▶ Execute

```
insert into itemvendas (idvenda,idproduto,quantidade,valorunitario,valoritem)  
values (2, 7, 4, 50.00, 200.00);
```

▶ Execute

```
insert into itemvendas (idvenda,idproduto,quantidade,valorunitario,valoritem)  
values (3, 6, 3, 5.00, 15.00);
```

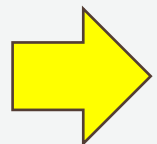
▶ Execute

```
insert into itemvendas (idvenda,idproduto,quantidade,valorunitario,valoritem)  
values (1, 6, 3, 5.00, 15.00);
```

Execute | JSON | Copy

```
select i.idproduto, p.descproduto, sum(i.quantidade)
  from itemvendas i
     inner join produto p
     on i.idproduto=p.idproduto
 group by i.idproduto, p.descproduto;
```

idprodu	descprc	sum
2	detergent	6.000
4	arroz	2.000
1	papel higi	10.000
3	desinfetar	3.000
7	leite longa	16.000
5	feijao	2.000
6	creme de	9.000



iditer integer	idvenda integer	idproduto integer	quantidade numeric
5	2	6	3.000
9	3	6	3.000
10	1	6	3.000

Agrupamento

Quantidade de
produtos vendidos
(agrupamento)

Agrupamento

```
▷ Execute | SQL | Copy  
select i.idproduto, p.descproduto, sum(i.quantidade)  
from itemvendas i  
inner join produto p  
on i.idproduto=p.idproduto  
group by i.idproduto, p.descproduto;
```

Quando utilizamos o Group by para realizarmos um agrupamento por algum campo (pivô), os campos escolhidos tem que estar exatamente iguais em group by e na exibição dos dados.

Execute | JSON | Copy

```
select i.idproduto, p.descproduto, sum(i.quantidade)
  from itemvendas i
    inner join produto p
      on i.idproduto=p.idproduto
 group by i.idproduto, p.descproduto
order by p.descproduto desc; 3ms
```

Para ordenarmos a saída de dados basta adicionar o order by após o group by.

Agrupamento

idprc	descproduto	sum
1	papel higienico	10.000
7	leite longa vida	16.000
5	feijao	2.000
2	detergente	6.000
3	desinfetante	3.000
6	creme de leite	9.000
4	arroz	2.000

Filtrando em um agrupamento.

- A cláusula GROUP BY possui como auxiliar, a cláusula **HAVING**;
- Estabelece para a cláusula GROUP BY as condições de saída;
- De forma análoga, a cláusula HAVING é semelhante à cláusula WHERE usada com o comando SELECT.

Agrupamento

```
select i.idproduto, p.descproduto, sum(i.quantidade)
  from itemvendas i
    inner join produto p
      on i.idproduto=p.idproduto
 group by i.idproduto, p.descproduto
having sum(i.quantidade) >= 9
order by p.descproduto desc; 1ms
```

idprc	descproduto	sum
1	papel higienico	10.000
7	leite longa vida	16.000
6	creme de leite	9.000

Suponha que deseja-se conhecer apenas os produtos que venderam 9 ou mais unidades.

Temos que realizar o group by para somarmos a quantidade por produto e depois filtrarmos o resultado

Filtrando em um agrupamento → Clausula WHERE em um Group by.

- Como vimos o filtro para uma informação agrupada é a clausula Having.
- Podemos utilizar a clausula Where, mas ela tem a função de realizar um filtro antes de ser realizado o agrupamento;
- Deve ser utilizada antes do agrupamento.

Agrupamento

```
select i.idproduto, p.descproduto, sum(i.quantidade)
  from itemvendas i
     inner join produto p
        on i.idproduto=p.idproduto
 WHERE p.perecivel = 'S'
 group by i.idproduto, p.descproduto
 having sum(i.quantidade) >= 9
 order by p.descproduto desc;
```

idproduto	descproduto	sum
1	papel higienico	10.000

- Em relação a última consulta, nesta, antes de realizarmos o agrupamento realizamos um filtro na tabela utilizando a cláusula **Where**;
- Deste modo os dados utilizados no processo de group by foi otimizado pois recebeu um pacote de dados menor.

Combinação de Consultas



Combinação de Consultas

Pode-se combinar os resultados de duas consultas utilizando as operações de:

- União (UNION)
- Interseção (INTERSECT)
- Exceção (EXCEPT)

Combinação de Consultas

Sintaxe:

select1 ... UNION [ALL] select2 ...

select1 ... INTERSECT [ALL] select2 ...

select1 ... EXCEPT [ALL] select2 ...

As operações de conjuntos podem ser aninhadas ou encadeadas. Por exemplo:

select1 ... **UNION** select2 ... **UNION** select3 ...

Combinação de Consultas

Crie a tabela

```
CREATE TABLE fornecedor (  
    idfornecedor integer,  
    nomefornecedor varchar(100),  
    estado varchar(02)  
);
```

Combinação de Consultas

Insira os dados

```
insert into fornecedor (idfornecedor,nomefornecedor,estado)
values (1, 'fornecedor 01', 'SP');
```

▷ Execute

```
insert into fornecedor (idfornecedor,nomefornecedor,estado)
values (2, 'fornecedor 02', 'SP');
```

▷ Execute

```
insert into fornecedor (idfornecedor,nomefornecedor,estado)
values (3, 'fornecedor 03', 'MG');
```

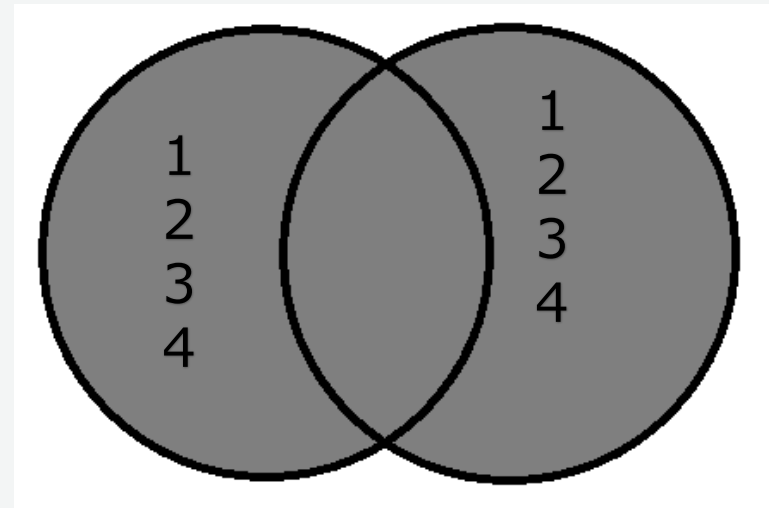
▷ Execute

```
insert into fornecedor (idfornecedor,nomefornecedor,estado)
values (4, 'fornecedor 04', 'MS');
```

Combinação de Consultas

- **UNION**
- UNION anexa o resultado do select2 ao resultado do select1 (embora não haja garantia que esta seja a ordem que as linhas realmente retornam).

Cliente fornecedor



Combinação de Consultas

Vamos unir os dados da tabela cliente e fornecedor

▶ Execute | JSON

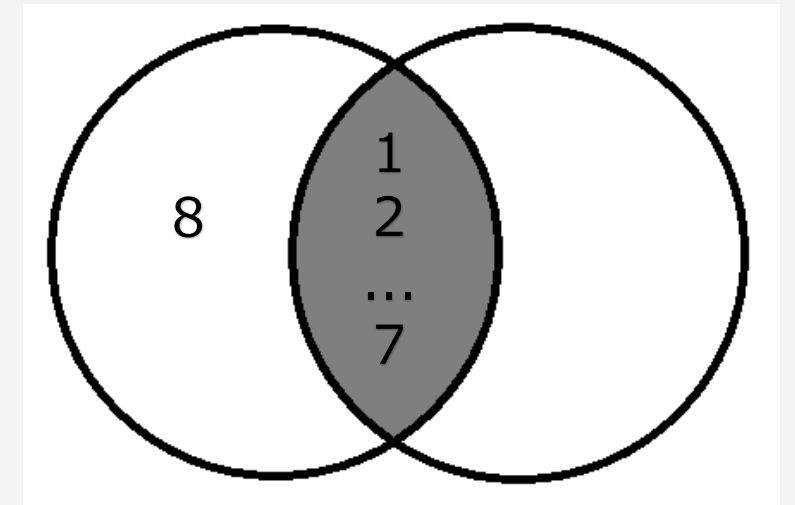
```
select idcliente as id, nomecliente as nome, estado from cliente  
UNION  
select idfornecedor as id, nomefornecedor as nome, estado from fornecedor  
order by nome; 7ms
```

id	nome	estado
1	fornecedor 01	SP
2	fornecedor 02	SP
3	fornecedor 03	MG
4	fornecedor 04	MS
2	Joao	SP
5	Joaquina	MG
3	Jose	MG
4	Maria	MS

Combinação de Consultas

- **INTERSECT**
- INTERSECT retorna todas as linhas presentes tanto no resultado do select1 quanto no resultado do select2.
- Para testar insira mais um produto a tabela:

Produto Itemvendas



```
insert into produto (idproduto, descproduto, saldoestoque, perecivel, valorunitario, datavalidade)
values (8, 'produto 08', 10, 'S', 40, '2025-01-20');
```

Combinação de Consultas

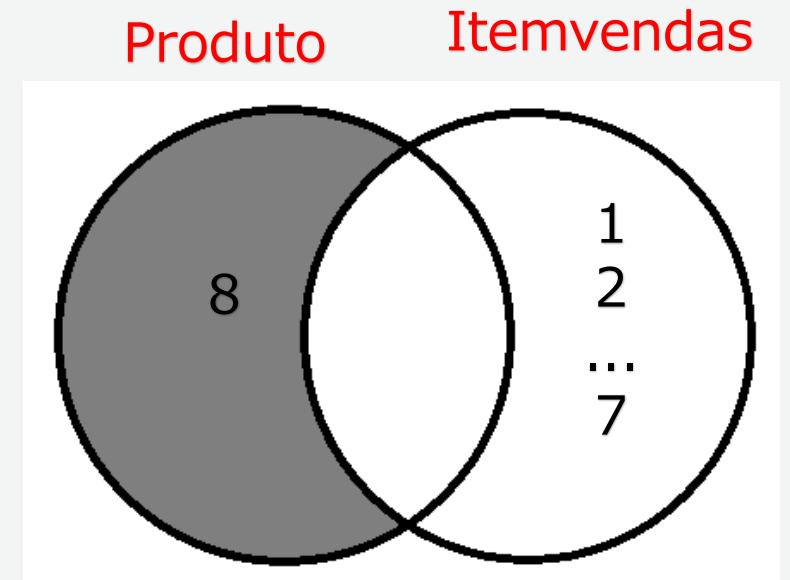
Vamos fazer a intersecção dos dados da tabela produto e itemvendas

```
select p.idproduto from produto p  
INTERSECT  
select i.idproduto from itemvendas i
```

idproduto
3
5
4
6
2
7
1

Combinação de Consultas

- **EXCEPT**
- EXCEPT retorna todas as linhas presentes no resultado do select1, mas que não estão presentes no resultado do select2 (às vezes isto é chamado de diferença entre duas consultas).



Combinação de Consultas

Vamos fazer a EXCEPT dos dados da tabela produto e itemvendas

```
select p.idproduto from produto p  
EXCEPT  
select i.idproduto from itemvendas i
```

idproduto ⇅

8

Consultas Aninhadas



Consultas Aninhadas

- O resultado de uma consulta é utilizado por outra consulta, de forma encadeada e no mesmo comando SQL
- O resultado do comando SELECT mais interno (subselect) é usado por outro SELECT mais externo para obter o resultado final

Consultas Aninhadas

- Subconsultas devem ser escritas entre parênteses ()
- Existem 3 tipos de subconsultas
 - ESCALAR → Retornam um único valor
 - ÚNICA COLUNA → Retornam várias linhas, mas apenas uma única coluna é obtida

Consultas aninhadas

```
SELECT  select_list  
FROM    tabela  
WHERE   operador expr
```

```
(SELECT  select_list  
FROM    tabela);
```

- Exemplo - Escalar:

Consultas Aninhadas

```
select * from venda v  
where idcliente = (select idcliente from cliente c where c.estado = 'MS');
```

idcliente
integer

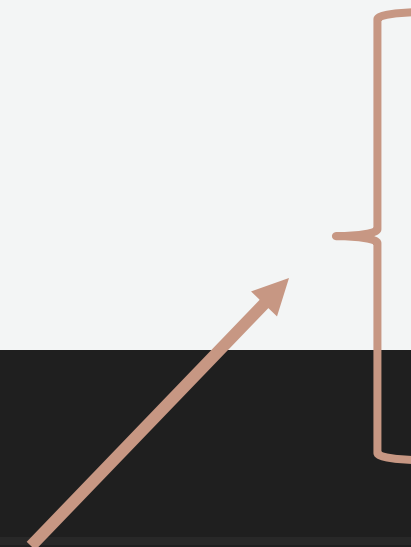
4

idvenda	idcliente	enderecoentrega
3	4	rua 4

- Exemplo – Tabela:

Consultas Aninhadas

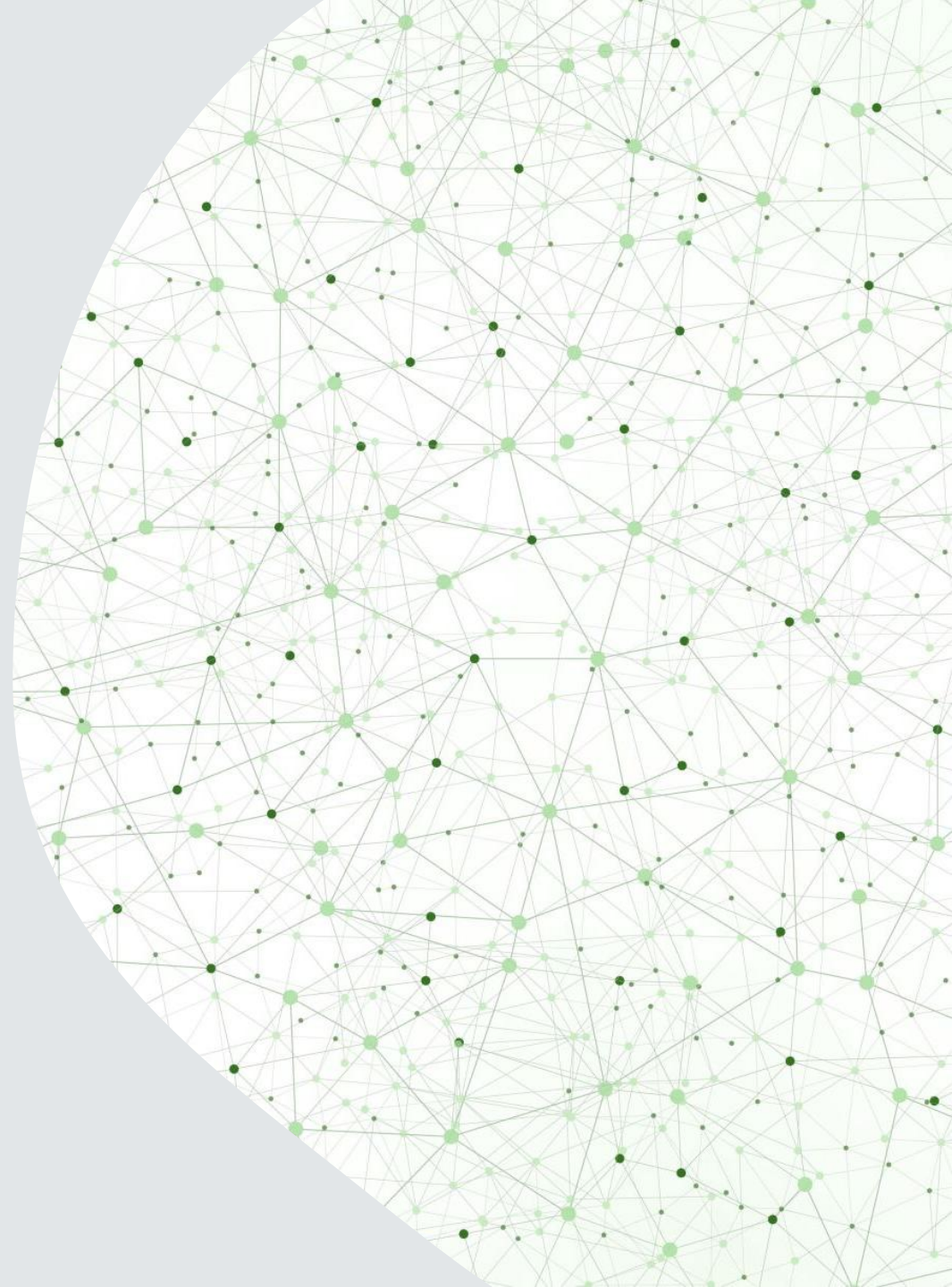
```
select * from itemvendas i
where i.idproduto in
    (select idproduto from produto where perecivel='S');
```



idprodu
integer
1
2
4
8

iditem	idvenda	idproduto	quantidade	valorunitario	valoritem
1	1	1	10.000	12.000	120.000
2	1	2	6.000	1.500	9.000
4	1	4	2.000	14.000	28.000

Ranqueamento Rank()



Função Rank() - Básico

- A função rank() é uma das funções de janela em SQL. As funções olham parte dos dados e computam os resultados para esta parte.
- Atribui uma classificação a cada linha com base em uma coluna fornecida.
- RANK() está incluída na declaração SELECT com a seguinte sintaxe:

```
RANK() OVER (ORDER BY column ASC|DESC)
```

Função Rank() - Básico

- Crie a seguinte estrutura no seu banco de dados:

```
CREATE TABLE animal_grupo (  
    id serial PRIMARY KEY,  
    nome VARCHAR (255) NOT NULL  
);
```

▷ Run | ⚡ Reset

```
INSERT INTO animal_grupo (nome)  
VALUES  
    ('Terrestre'),  
    ('Aquatico'),  
    ('Aereo');
```

```
CREATE TABLE animal (  
    id serial PRIMARY KEY,  
    nome VARCHAR (255) NOT NULL,  
    tempovida DECIMAL (11, 2),  
    animal_id INT NOT NULL,  
    FOREIGN KEY (animal_id) REFERENCES animal_grupo (id)  
);
```

▷ Run | ⚡ Reset

```
INSERT INTO animal(nome, animal_id, tempovida)  
VALUES  
    ('vaca', 1, 10),  
    ('cachorro', 1, 7),  
    ('boi', 1, 13),  
    ('lobo', 1, 11),  
    ('baleia azul', 2, 80),  
    ('golfinho', 2, 5),  
    ('cavalo marinho', 2, 3),  
    ('polvo', 2, 8),  
    ('morcego', 3, 4),  
    ('esquilos voadores', 3, 1),  
    ('Petauro', 3, 2);
```

Função Rank() - Básico

- Exemplo da função RANK().

```
SELECT
  id,
  nome,
  tempovida,
  RANK () OVER (
    ORDER BY tempovida DESC
  ) rank_tempovida
FROM
  animal;
```

* id integer	* nome varchar(255)	tempovida numeric(11,2)	rank_tempovida
Filter	Filter	Filter	Filter
5	baleia azul	80.00	1
3	boi	13.00	2
4	lobo	11.00	3
1	vaca	10.00	4
8	polvo	8.00	5
2	cachorro	7.00	6
6	golfinho	5.00	7
9	morcego	4.00	8
7	cavalo marinho	3.00	9
11	Petauro	2.00	10
10	esquilos voadores	1.00	11

Função Rank() - Básico

- Agora crie uma consulta onde deverá ser fornecida as informações: descrição do produto e a quantidade vendida de cada produto ranqueadas como um indicador de vendas na ordem decrescente.

- Agora crie uma consulta onde deverá ser fornecida as informações: descrição do produto e a quantidade vendida de cada produto ranqueadas como um indicador de vendas na ordem decrescente.

- Resposta

```
select v.idproduto, p.descproduto,  
       sum(v.quantidade) quantidade_vendida,  
       RANK() over (order by sum(v.quantidade) desc)  
       as indicador_vendas  
from itemvendas v  
     inner join produto p  
       on v.idproduto = p.idproduto  
group by v.idproduto, p.descproduto;
```

Função Rank() - Básico

idproduto	descproduto	quantidade_vendida	indicador_vendas
Filter	Filter	Filter	Filter
2	detergente	12.000	1
7	leite longa vida	12.000	1
1	papel higienico	10.000	3
3	desinfetante	6.000	4
6	creme de leite	3.000	5
4	arroz	2.000	6
5	feijao	2.000	6

- Agora atualize seu banco de dados novamente.

Função Rank() - Básico

```
create table grupoproduto (  
    id serial PRIMARY KEY,  
    descricao varchar(100)  
);
```

▷ Run | ⚡ Reset

```
insert into grupoproduto (descricao)  
VALUES ('Alimento'), ('Higiene Pessoal'), ('Limpeza')
```

▷ Run | ⚡ Reset

```
alter table produto add column idgrupoproduto int;
```

▷ Run | ⚡ Reset

```
update produto set idgrupoproduto = 1 where idproduto in (4,5,6,7);
```

▷ Run | ⚡ Reset

```
update produto set idgrupoproduto = 2 where idproduto = 1;
```

▷ Run | ⚡ Reset

```
update produto set idgrupoproduto = 3 where idproduto in (2,3);
```

Função Rank() com Partition by

- A função rank() é usada para atribuir uma classificação a cada linha do conjunto de resultados da consulta na partição especificada.
- A classificação da primeira linha em cada partição é 1.

- Sintaxe:

Syntax:

```
RANK() OVER (  
    [PARTITION BY partition_expression, ... ]  
    ORDER BY sort_expression [ASC | DESC], ...  
)
```

- Primeiro a cláusula PARTITION BY cria uma partição no resultado da consulta onde a função RANK() é aplicada.
- A cláusula ORDER BY define a ordem em que os resultados da consulta são exibidos.

- Exemplo:

```
SELECT
  g.nome as grupo,
  a.nome,
  a.tempovida,
  RANK () OVER (
    PARTITION BY g.id
    ORDER BY tempovida DESC
  ) rank_tempovida
FROM
  animal a
  inner join animal_grupo g
  on a.animal_id = g.id;
```

Função Rank() com Partition by

grupo	nome	tempovida	rank_tempovida
Filter	Filter	Filter	Filter
Terrestre	boi	13.00	1
Terrestre	lobo	11.00	2
Terrestre	vaca	10.00	3
Terrestre	cachorro	7.00	4
Aquatico	baleia azul	80.00	1
Aquatico	polvo	8.00	2
Aquatico	golfinho	5.00	3
Aquatico	cavalo marinho	3.00	4
Aereo	morcego	4.00	1
Aereo	Petauro	2.00	2
Aereo	esquilos voadores	1.00	3

Função Rank() com Partition By

- Agora crie uma consulta onde deverá ser fornecida as informações: grupo do produto, descrição do produto, quantidade total vendida por produto.
- As informações deverão estar particionadas pelo grupo de produto e onde deve-se criar um ranqueamento das vendas dos produtos.

- Resposta

Função Rank() com Partition By

```
select v.idproduto, g.descricao as grupoproduto,
       p.descproduto, sum(v.quantidade)
       as quantidade_vendida,
       RANK() OVER (
         PARTITION BY g.descricao
         ORDER BY sum(v.quantidade) desc)
       as indicador_vendas
from itemvendas v
  inner join produto p
    on v.idproduto = p.idproduto
  inner join grupoproduto g
    on p.idgrupoproduto = g.id
group by v.idproduto, g.descricao, p.descproduto;
```

idproduto	grupoproduto	descproduto	quantidade_vendida	indicador_vendas
Filter	Filter	Filter	Filter	Filter
7	Alimento	leite longa vida	12.000	1
6	Alimento	creme de leite	3.000	2
5	Alimento	feijao	2.000	3
4	Alimento	arroz	2.000	3
1	Higiene Pessoal	papel higienico	10.000	1
2	Limpeza	detergente	12.000	1
3	Limpeza	desinfetante	6.000	2